SIMATS ENGINEERING

ASSESSMENT TOOL 2 — DATA INTERPRETATION (MEDIUM)

Course Name	Internet Programming	**Course Code**	CSA43
Student Name	Naveen.S	**Register No.**	192324248
Course Outcome	CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery.	**Weightage**	20%

1. HTTP Response Analysis

Given: HTTP/1.1 200 OK; Date: Mon, 01 Apr 2026 10:00:00 GMT; Content-Type: text/html; Content-Length: -150; Connection: keep-alive.

1. Incorrect or invalid fields

The main invalid field is **Content-Length: -150**. Content-Length specifies the size of the response body in bytes and cannot be negative. The date also appears inconsistent: 1 April 2026 was a Wednesday, not Monday. The date should be checked and corrected. The other listed fields are syntactically plausible.

2. Issue with Content-Length and its impact

A negative content length is invalid because a message body cannot have a negative number of bytes. A client may reject the response, treat the header as malformed, or handle the message inconsistently. This can cause parsing errors or a failed page load. The value must be the actual non-negative number of bytes in the response body.

3. Does the status code match the response correctness?

200 OK means the request was successfully handled. However, the invalid Content-Length makes the response header malformed, so the response is not fully correct even though the status code says success. A server should send a valid response header and body; if it cannot fulfil the request, it should return an appropriate error status instead.

4. Suggested corrections

Replace the negative length with the actual body size, for example `Content-Length: 150` only if the body is exactly 150 bytes. Correct the date to the actual date and weekday, and ensure the body length matches the header. Keep `200 OK` if the request succeeded and the response is valid.

2. DOM Structure Analysis

1. Missing or improperly closed elements

The <p> element containing “HelloWorld” is not explicitly closed before the `</div>`. The paragraph should be closed with `</p>` before the div is closed.

2. How the browser interprets the structure

HTML browsers use error-recovery rules to construct a DOM even when markup is imperfect. The browser will generally infer the end of the paragraph when it encounters the closing div, so the page may still display. However, relying on automatic recovery can make the resulting DOM harder to predict and maintain.

3. Impact on rendering and layout

The text may appear normally, but the browser may repair the structure automatically. In more complex pages, invalid nesting can affect spacing, CSS selectors, JavaScript DOM traversal, accessibility tools, and consistency between browsers.

4. Corrected DOM/HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

3. GET vs POST in Login System

1. Differences in data transmission

GET commonly sends data in the URL as query parameters, such as `/login?user=admin&pass=1234`. These values can appear in browser history, server logs, analytics, and copied links. POST sends submitted form data in the request body, so credentials are not displayed in the URL. POST does not by itself encrypt the data.

2. More suitable method for login

POST is more suitable for submitting login credentials because it avoids placing the username and password in the URL. The login request must also use HTTPS, which encrypts the request in transit. Authentication should be performed securely on the server.

3. Security risks of GET for authentication

Credentials in a GET URL can be stored in browser history and logs, exposed through screenshots or copied links, and potentially leak through referrer information. URLs may also be cached or recorded by monitoring systems. These risks make GET unsuitable for sending passwords.

4. Best practices for secure login

Use HTTPS; submit credentials using POST; never store passwords in URLs; store passwords on the server using a strong salted password-hashing algorithm; validate input and apply rate limiting; use secure, HttpOnly, SameSite session cookies; protect against CSRF where relevant; and avoid revealing whether the username or password alone was incorrect.

4. HTML Code Inspection

1. Missing closing tags

The `<h1>` heading and `<p>` paragraph are not explicitly closed. They should end with `</h1>` and `</p>`. The `<img>` element is a void element in HTML and does not need a closing tag.

2. Issues with the img tag

The image tag has a `src` attribute, but it lacks an `alt` attribute. Alternative text supports screen-reader users and appears if the image cannot be displayed. The file `image.jpg` must also exist at the referenced path; otherwise, the image will fail to load.

3. How browsers handle these errors

Browsers typically repair omitted end tags and continue rendering the document. The heading and paragraph may still appear, but malformed markup can lead to inconsistent DOM structure. If the image path is wrong, the browser displays a broken-image indicator or no image; without alt text, users may not know what the image represents.

4. Corrected HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Description of the image">
</body>
</html>
```

5. Browser–Server Communication Flow

1. Interpretation of each step

Step 1 — User enters a URL: The browser identifies the requested scheme, domain, and resource path.
Step 2 — DNS lookup: The browser or operating system finds the IP address associated with the domain, often using a cache first.
Step 3 — HTTP request is sent: The browser establishes the required connection (and TLS for HTTPS) and sends a request containing a method, path, headers, and sometimes a body.
Step 4 — Server processes the request: The server routes the request, runs application logic, may query a database, and prepares a response.
Step 5 — Response is returned: The server sends a status code, headers, and usually a response body.
Step 6 — Browser renders the page: The browser parses HTML, fetches referenced resources such as CSS, JavaScript, and images, builds the page structure, and paints the result.

2. Where delays can occur

Delays can occur during DNS resolution, connection establishment, TLS negotiation, network transmission, server-side processing, database queries, downloading large resources, and browser parsing or JavaScript execution. Congestion, packet loss, slow hosting, and unoptimized files can increase total loading time.

3. Role of DNS

DNS translates a human-readable domain name (for example, `example.com`) into an IP address that computers use to locate the server. DNS caching reduces repeated lookups. If DNS is slow or fails, the browser may wait longer or may not reach the server at all.

4. Performance optimizations

Use DNS caching and a reliable DNS provider; enable HTTPS with efficient connection reuse; use a CDN for static assets; compress text resources; optimize images; minify CSS and JavaScript; cache suitable resources; reduce unnecessary HTTP requests; optimize server code and database queries; and defer non-critical JavaScript. Monitor real-user performance to identify the main bottlenecks.

Declaration

I, **Naveen.S** (Register No. **192324248**), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Student Signature: _____